

FIGMA GUIDE

Table of Contents

1. Core Concepts
 2. Auto Layout: The Foundation
 3. Components: Building Reusable Widgets
 4. Component Properties: Making Components Flexible
 5. Pages: Organizing Your Workspace
 6. Building Buttons
 7. Building Input Fields
 8. Building Complex Interactions
 9. Designing for AI Code Generators
 10. Responsive Design Patterns
 11. Bottom Navigation & Tab Systems
 12. Complete Execution Checklist
-

Core Concepts

Why Figma is Different from Drawing Tools

Traditional design tools like Photoshop treat your design as pixels. Figma treats your design as **code-like data structures** that can be read, parsed, and automatically converted into actual code.

When an AI code generator (like Antigravity) reads your Figma file via an MCP server, it doesn't see the visual pixels. It reads a massive JSON structure underneath that looks something like this:

```
{
  "name": "LoginButton",
  "type": "COMPONENT",
  "children": [
    {
      "name": "Label",
      "type": "TEXT",
      "content": "Sign In",
      "fontFamily": "Inter",
      "fontSize": 16
    }
  ],
  "layoutMode": "HORIZONTAL",
  "padding": 16,
  "fills": [{ "color": "#007AFF" }]
}
```

This is why structure matters more than aesthetics. The AI reads this JSON, not your beautiful design. If your file is full of "Rectangle 42" and unorganized layers, the AI hallucinates. If your file is logically structured with semantic names, the AI generates perfect code.

The Three Mental Models You Must Understand

1. Frames vs. Components vs. Instances

- **Frame (田):** A rectangular container. Use this for layout grouping, sections, and one-off structural elements. It's a "dumb box."
- **Main Component (❖):** A reusable blueprint. When you press **Ctrl + Alt + K**, you create a Main Component. This is like defining a class or function in code. The AI reads the **type: COMPONENT** flag and knows to generate a reusable widget.
- **Instance (◇):** A copy of a Main Component. When you drag from the Assets panel, you're creating an Instance. This is like calling a function with specific parameters. If you change the Main Component, all Instances update automatically.

Why This Matters:

- If you use the Main Component directly in your design, the AI thinks you're defining the class inline in your layout code. Messy.
- If you use Instances, the AI sees "call this widget here with these properties." Clean.

2. Auto Layout: Flexbox for Designers

Auto Layout is Figma's version of flexbox, CSS Grid, or the **Row/Column** layouts in Flutter.

When you press **Shift + A** on a frame or text, you're saying: "Turn this into a flex container. Make the children automatically space themselves based on padding, gaps, and alignment rules."

Why This Matters:

- Without Auto Layout, the AI has no way to know your layout intent. It sees absolute coordinates and tries to generate CSS with hardcoded pixel positions.
- With Auto Layout, the AI reads **layoutMode: HORIZONTAL**, **gap: 16**, and immediately generates clean flexbox code.

3. Properties: The Bridge Between Design and Code

Component Properties are like function parameters. When you add a Text Property to a button, you're saying: "This text is dynamic. Developers will pass in different values."

Why This Matters:

- Without properties, the text is hardcoded into your button. The AI generates a button that always says "Login."
- With properties, the button can say "Login," "Sign Up," "Logout" depending on what property value you set on each Instance.

Auto Layout: The Foundation

What is Auto Layout?

Auto Layout is a system that automatically sizes and positions elements based on rules you define. Think of it as "responsive design at the design stage."

The Three Resizing Options

When you apply Auto Layout to a frame, every child element and the frame itself has a **Horizontal Resizing** and **Vertical Resizing** setting:

1. Hug (Shrink-wrap)

The container sizes itself based on its contents.

```
Button Text: "Click Me"  
Button Size: 120px × 48px
```

```
Change Text: "Click Me With A Really Long Label"  
Button Size: 280px × 48px (Auto-expanded!)
```

When to use: Buttons, badges, chips. Anything where the content drives the size.

Code translation: Width and height are `auto`, content determines the size.

2. Fill Container (Stretch)

The container stretches to fill available space from its parent.

```
Screen Width: 400px  
Parent Container: Fill (stretches to 400px)  
Child Button: Fill (stretches to 100% of parent, minus gaps)  
Result: Button takes 50% of the screen width
```

When to use: Elements that should stretch to full width, responsive layouts.

Code translation: `width: 100%` in CSS, or `flex: 1` in Flutter.

3. Fixed

The container stays exactly the size you specify, no matter what.

```
Button Width: 120px (stays 120px forever, even if parent shrinks)
```

When to use: Rarely. Usually only for specific visual designs where you need pixel-perfect control.

Code translation: `width: 120px` in CSS.

Auto Layout Deep Dive: Building a Button

Let's say you want to build a button. Here's the **exact sequence**:

Step 1: Type the Text

```
Press T → Click canvas → Type "Submit"
```

Step 2: Apply Auto Layout

```
Select the text → Press Shift + A  
Result: A frame is created around the text, hugging it tightly.
```

Step 3: Adjust Padding

Right panel → Auto Layout section → Horizontal Padding: 16px, Vertical Padding: 12px
Result: The frame now has internal space (16px left/right, 12px top/bottom)

Step 4: Style It

Fill: Add a blue background
Stroke: Add a border (optional)
Corner Radius: Set to 8px for rounded corners

What's happening under the hood:

- The frame has `layoutMode: HORIZONTAL` (or VERTICAL depending on your content)
- It has `paddingLeft: 16`, `paddingRight: 16`, `paddingTop: 12`, `paddingBottom: 12`
- Children are centered with `alignItems: CENTER`
- The frame itself is set to `Hug contents` width and height

Why this matters for AI:

When Antigravity reads this button, it sees the padding and alignment rules. Instead of trying to position the text absolutely or dragging it manually (which the AI can't do), the AI generates:

```
.button {  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  padding: 12px 16px;  
  background-color: #007AFF;  
  border-radius: 8px;  
}
```

CSS

Perfect, semantic, production-ready code. All because you used Auto Layout instead of dragging.

The Golden Rule of Auto Layout

Always use Auto Layout for anything that contains other elements. If your frame contains text, icons, or other shapes, apply Auto Layout. The one exception is complex vector graphics (logos, illustrations) where you want absolute control over paths.

Components: Building Reusable Widgets

Why Components Exist

Imagine you need to use the same button 50 times across your entire design file. Without components, you have 50 separate buttons. If you decide to change the color from blue to green, you have to manually change all 50.

Components solve this: You change the **Main Component** once, and all 50 **Instances** update automatically.

Code Analogy:

```
JS
// Without Components (bad):
// Defining the button 50 times
<button style={buttonStyle1}>Login</button>
<button style={buttonStyle1}>Sign Up</button>
<button style={buttonStyle1}>Logout</button>
// ... 47 more times

// With Components (good):
// Define once
const Button = ({ label, color }) => (
  <button style={{ backgroundColor: color }}>
    {label}
  </button>
);

// Use many times
<Button label="Login" color="blue" />
<Button label="Sign Up" color="blue" />
<Button label="Logout" color="blue" />
// Change color in one place: color becomes "green" everywhere
```

Creating Your First Component

Scenario: You're building a mobile app with buttons that appear on multiple screens.

Step 1: Design the Button (No Component Yet)

```
Press T → Type "Button"
Select text → Shift + A (Auto Layout)
Add padding: 16px horizontal, 12px vertical
Add fill: blue background
Add corner radius: 8px
Name the frame: "Button"
```

Step 2: Make it a Main Component

Select the frame → Press Ctrl + Alt + K

OR: Right-click → Create Component

Result: The frame's icon changes from  to  (solid purple diamond)
The frame's outline becomes purple.

Step 3: Use Instances (Never Use the Main Component Directly)

Open Assets panel on the left (folder icon)

Find your  Button

Drag it onto your screen MULTIPLE times

Notice the icon is now  (hollow purple diamond) - these are Instances

Step 4: Modify Instances Without Affecting the Blueprint

Select an Instance on your screen

In the right panel, you can override properties:

- Change text: "Login"

- Change colors: green instead of blue

These changes only affect this one Instance.

The Main Component stays blue.

Step 5: Change the Blueprint, Watch All Instances Update

Go to the Components page

Select your Main Component 

Change the corner radius from 8px to 12px

Jump back to your screen: ALL instances now have 12px corner radius!

Why You Should Never Use the Main Component Directly

Imagine you drag your Main Component  Button directly onto your LoginPage and don't create an Instance.

What happens:

1. You later decide to redesign that page completely.
2. You delete the entire LoginPage frame.
3. **Your Main Component is deleted too.** It was inside that frame!
4. Now every other screen that tried to use this button is broken.

If you had used Instances:

1. You delete the LoginPage.

2. The Instances are deleted, but the Main Component is safe in your Components page.
3. You can always create new Instances on new screens.

Best Practice Workflow:

1. Create Main Component on a dedicated "Components" or "Design System" page
2. Drag Instances from the Assets panel into your actual design pages
3. Never touch the Main Component except to update its design
4. Instances live in your actual screens and handle all the variation

Component Properties: Making Components Flexible

What are Component Properties?

Component Properties are like function parameters. They allow you to create one Main Component that can behave differently depending on what value you pass in.

Code Analogy:

```
JS
// Without Properties:
const LoginButton = () => <button>Login</button>;
const SignUpButton = () => <button>Sign Up</button>;
// Need 50 different button components for 50 different texts

// With Properties:
const Button = ({ label, variant, hasIcon }) => (
  <button className={variant} style={{ paddingLeft: hasIcon ? 12 : 16 }}>
    {hasIcon && <Icon />}
    {label}
  </button>
);
// One component, infinite variations
```

Types of Component Properties

1. Text Property (For Dynamic Text)

Use Case: A button that can say "Login," "Sign Up," or "Logout" depending on context.

How to Create:

Step 1: Create your Main Component ♦ Button with text inside
Step 2: Select the Main Component
Step 3: Right panel → Properties section → Click + → Select "Text"
Step 4: Name it "label" (semantic name)
Step 5: Set default value: "Submit"
Step 6: Double-click inside the component to select the actual text layer
Step 7: Right panel → Text section → Click the hexagon icon next to the text
Step 8: Select "Create text property" and link it to your "label" property

Result:

- The Main Component now has a property called `label`
- When you drag an Instance, you can change the text in the right panel
- The AI reads this as a dynamic, prop-driven component

Code Translation:

```
<Button label="Login" />  
<Button label="Sign Up" />  
<Button label="Logout" />  
// All using the same Button component, different labels
```

JS

2. Boolean Property (For Toggling Elements On/Off)

Use Case: A button that sometimes has an icon and sometimes doesn't.

How to Create:

Step 1: Create your Main Component ♦ Button with an icon inside
Step 2: Select the Main Component
Step 3: Right panel → Properties → Click + → Select "Boolean"
Step 4: Name it "showIcon"
Step 5: Leave default value as true
Step 6: Double-click inside and select the icon layer
Step 7: In the Appearance section on the right, find the eye icon (visibility)
Step 8: Click the hexagon next to it and link to your "showIcon" property

Result:

- When you drag an Instance, a toggle appears in the right panel
- Toggle it off, the icon disappears from that Instance
- The Main Component still has the icon
- The AI reads this as conditional rendering

Code Translation:

```
<Button label="Sign In" showIcon={true} icon={<GoogleLogo />} />
<Button label="Continue as Guest" showIcon={false} />
// Same button, different visibility of icon
```

3. Instance Swap Property (For Swapping Components)

Use Case: A card that can show different icon types (home, search, user).

How to Create:

```
Step 1: Create multiple icon components: ♦ HomeIcon, ♦ SearchIcon, ♦
UserIcon
Step 2: Create a card Main Component that contains a placeholder icon
Step 3: Select that placeholder icon Instance inside the card
Step 4: Right panel → Properties → Click + → Select "Instance"
Step 5: Name it "icon"
Step 6: This property now allows swapping the icon Component
```

Result:

- When you drag a card Instance, you can swap the icon in the right panel
- Select HomeIcon, the card shows a home icon
- Select SearchIcon, the icon changes instantly
- The AI reads this as component composition

Why Properties Matter for AI

Without properties, your button component is hardcoded:

```
{
  "name": "Button",
  "type": "COMPONENT",
  "children": [
    {
      "name": "Label",
      "type": "TEXT",
      "content": "Login" // HARDCODED!
    }
  ]
}
```

The AI has no choice but to generate a button that always says "Login."

With properties:

```
{
  "name": "Button",
  "type": "COMPONENT",
  "properties": [
    {
      "name": "label",
      "type": "TEXT",
      "defaultValue": "Login"
    }
  ],
  "children": [
    {
      "name": "Label",
      "type": "TEXT",
      "content": "{label}" // DYNAMIC!
    }
  ]
}
```

Now the AI generates:

```
const Button = ({ label = "Login" }) => (
  <button>{label}</button>
);
```

JS

A function with parameters. Reusable. Perfect.

Pages: Organizing Your Workspace

Why Pages Exist

A Figma file is like a notebook. Pages are the tabs or chapters inside it. They give you multiple separate, infinite canvases without creating entirely new files.

The Standard Page Structure

A professional Figma file looks like this:

 Cover
└ Single frame with project title, version, status

 Design System / Components
├ ♦ Button variants
├ ♦ InputField variants
├ ♦ Icons
└ Color palette, typography guide

 Wireframes
└ Low-fidelity mockups

 Work in Progress (WIP)
└ Screens you're actively designing

 Ready for Dev
└ Finalized screens approved for hand-off

 Archive
└ Old iterations, deprecated flows

 Sandbox / Playground
└ Experimentation area, no rules

Why Separate Components to Their Own Page?

Without separation:

```
LoginPage
├ TextArea
├ InputForm
├ ♦ Button (Main Component)  <- Confusing! Is this a design or a blueprint?
└ ActionContainer
```

With separation:

```
Components Page:
├ ♦ Button
├ ♦ InputField
└ ♦ TextButton

LoginPage:
├ TextArea
├ InputForm
├ ◇ Button (Instance)  <- Clear! This is a usage of the button
└ ActionContainer
```

The second is infinitely clearer. It tells everyone (and the AI) that Components is the "library" and LoginPage is where components are "consumed."

Building Buttons

Complete Button Building Guide

Scenario: Creating a Primary Button Component

Step 1: Design the Text

```
Press T on keyboard  
Click on canvas  
Type your button label: "Submit"
```

Step 2: Style the Text (Optional but Good Practice)

```
Select the text  
Right panel → Text section  
Font: Inter (or your app's font)  
Size: 16px  
Weight: Semi-bold (600)  
Color: White (since we'll have a blue background)
```

Step 3: Create Auto Layout Frame

```
With text selected → Press Shift + A  
Result: Frame created around text, hugging it  
  
Right panel → Auto Layout section:  
- Direction: Horizontal (default, for single text)  
- Horizontal Padding: 16px  
- Vertical Padding: 12px  
- Alignment: Center (X and Y center)
```

Step 4: Add Visual Styling

Right panel → Fill section:

- Click + to add fill
- Choose blue: #007AFF

Right panel → Stroke section:

- Click + to add stroke (optional)
- Color: darker blue
- Weight: 1px

Right panel → Corner Radius:

- Set to 8px for slightly rounded

Step 5: Create the Main Component

Select the frame → Press Ctrl + Alt + K

Result: Purple outline, icon changes to ♦

Rename it: "Button" or "PrimaryButton"

Step 6: Add the Text Property

With Main Component selected:

Right panel → Properties section → Click +

Select "Text"

Name it: "label"

Default value: "Submit"

Double-click inside component to select text layer

Right panel → Text section → Click hexagon icon

Link to "label" property

Step 7: Move to Components Page

Right-click on ♦ Button

Select "Move to page → Components"

(This keeps your Main Component safe and organized)

Step 8: Use on Your Screen

Open Assets panel (left sidebar)

Find your Button component

Drag it onto your LoginPage

It's now an Instance ◊

Click the Instance

Right panel → Change "label" property to "Login"

Drag another Instance, set label to "Sign Up"

Result:

❖ Button Main Component (on Components page)

└─ label: "Submit" (default)

LoginPage

└─ ◊ Button (Instance: label = "Login")

└─ ◊ Button (Instance: label = "Sign Up")

Building a Button with an Icon

Scenario: "Sign in with Google" Button

Step 1: Prepare the Google Logo

Import or design your Google logo

Flatten all vectors: Select → Right-click → Flatten

Make it a Component: Ctrl + Alt + K

Name it: "GoogleLogo"

Move to Components page

Step 2: Build the Button Layout

Step 1: Add Auto Layout Text: "Sign in with Google"

Step 2: Add the Logo: Drag ◊ GoogleLogo Instance next to text

Step 3: Select both → Shift + A

Step 4: Direction: Horizontal

Step 5: Gap: 8px (space between logo and text)

Step 6: Alignment: Center

Step 7: Padding: 16px horizontal, 12px vertical

Step 3: Make it a Component with Properties

Select frame → Ctrl + Alt + K
Name it: "SocialButton" or "ButtonWithIcon"

Right panel → Properties:

1. Add Text Property "label" (link to text layer)
2. Add Boolean Property "showIcon" (link to logo's visibility)

Step 4: Create Variants (Different States)

With Main Component selected:
Click "Add Variant" in toolbar

Property 1: Type

- WithIcon
- TextOnly

Property 2: Variant (Name it)

- Default
- Hover (optional)

Step 5: Design Each Variant

WithIcon + Default:

- ✓ Logo visible
- ✓ Text: "Sign in with Google"
- ✓ Background: Blue
- ✓ Text color: White

TextOnly + Default:

- ✗ Logo hidden
- ✓ Text: "Continue as Guest"
- ✓ Background: White
- ✓ Text color: Blue

What the AI Sees:


```
{
  "name": "ButtonWithIcon",
  "type": "COMPONENT",
  "properties": [
    {
      "name": "label",
      "type": "TEXT"
    },
    {
      "name": "type",
      "type": "VARIANT",
      "values": ["WithIcon", "TextOnly"]
    }
  ],
  "variants": [
    {
      "type": "WithIcon",
      "children": [
        { "name": "GoogleLogo", "visible": true },
        { "name": "Label", "content": "{label}" }
      ]
    },
    {
      "type": "TextOnly",
      "children": [
        { "name": "GoogleLogo", "visible": false },
        { "name": "Label", "content": "{label}" }
      ]
    }
  ]
}
```

The AI generates:

```
JS
const ButtonWithIcon = ({ label, type = "WithIcon" }) => {
  return (
    <button>
      {type === "WithIcon" && <GoogleLogo />}
      <span>{label}</span>
    </button>
  );
};
```

Perfect, production-ready component.

Building Input Fields

Complete Input Field Guide

Scenario: A Modern Text Input with Error States

Step 1: Add the Placeholder Text

Press T → Click canvas → Type "Enter email..."
Select text
Right panel → Text:
- Font: Inter, 16px
- Color: #999999 (light gray for placeholder look)

Step 2: Create the Auto Layout Frame

Select text → Shift + A
Right panel → Auto Layout:
- Direction: Horizontal (left-to-right)
- Padding: 16px horizontal, 12px vertical
- Alignment: Left (text should align left, not center)

Step 3: Style the Input Box

Right panel → Fill:
- Color: #F5F5F5 (light gray background)

Right panel → Stroke:
- Color: #E0E0E0 (subtle gray border)
- Weight: 1px

Right panel → Corner Radius:
- Set to 4px or 8px

Step 4: Make it a Main Component

Select frame → Ctrl + Alt + K
Name it: "TextInput" or "InputField"

Step 5: Add the Text Property

Right panel → Properties → Click +
Select "Text"
Name it: "placeholder"
Default: "Enter email..."

Double-click inside → Select text layer
Right panel → Text → Click hexagon → Link to "placeholder" property

Step 6: Create Input States with Variants

With Main Component selected:
Click "Add Variant" button

Property: "State"
Create three variants:
1. State: Default
2. State: Focused
3. State: Error

Step 7: Design Each State

Default State:

Border: #E0E0E0 (light gray)
Border Width: 1px
Background: #F5F5F5
Text Color: #999999

Focused State:

(Same as Default, but:)
Border: #007AFF (blue)
Border Width: 2px (thicker to show focus)
Background: #FFFFFF (white)
Text Color: #333333 (darker)
(Optional: Add a subtle blue shadow)

Error State:

Border: #FF3B30 (red)
Border Width: 2px
Background: #FFE0E0 (very light red)
Text Color: #FF3B30 (red)
(Optional: Add error message text below)

Step 8: Move to Components Page

Right-click → Move to page → Components

Step 9: Use on Your Screen

Open Assets panel

Drag ◇ TextInput Instance onto LoginPage

Set placeholder property to "Email Address"

Drag another Instance below it

Set placeholder property to "Password"

(This one might use a different library component for password masking, but same structure)

What the AI Sees:

```

{
  "name": "TextInput",
  "type": "COMPONENT",
  "properties": [
    {
      "name": "placeholder",
      "type": "TEXT"
    },
    {
      "name": "state",
      "type": "VARIANT",
      "values": ["Default", "Focused", "Error"]
    }
  ],
  "variants": [
    {
      "state": "Default",
      "fills": [{ "color": "#F5F5F5" }],
      "strokes": [{ "color": "#E0E0E0", "width": 1 }]
    },
    {
      "state": "Focused",
      "fills": [{ "color": "#FFFFFF" }],
      "strokes": [{ "color": "#007AFF", "width": 2 }]
    },
    {
      "state": "Error",
      "fills": [{ "color": "#FFEBEE" }],
      "strokes": [{ "color": "#FF3B30", "width": 2 }]
    }
  ]
}

```

The AI generates:

```
const TextInput = ({ placeholder, state = "Default" }) => {  
  const styles = {  
    Default: { borderColor: "#E0E0E0", background: "#F5F5F5" },  
    Focused: { borderColor: "#007AFF", background: "#FFFFFF" },  
    Error: { borderColor: "#FF3B30", background: "#FFE0E0" }  
  };  
  
  return (  
    <input  
      placeholder={placeholder}  
      style={styles[state]}  
      onFocus={() => /* handle focus */}  
    />  
  );  
};
```

Building Complex Interactions

Making Things Interactive: Prototyping

Scenario: Button that Navigates to Another Frame

Step 1: Set Up Your Screens

```
Create Frame A (LoginPage)  
├ Title: "Welcome Back"  
├ InputField (email)  
├ InputField (password)  
└ ◇ Button: label="Login"
```

```
Create Frame B (HomePage)  
├ Title: "Welcome Home"  
└ Greeting text
```

Step 2: Switch to Prototype Mode

Top right of Figma window → Click "Prototype" tab
(Next to "Design" tab)

Step 3: Create the Connection

Click on your Button Instance in LoginPage

You'll see a small circle appear on the right edge of the button
Click and drag that circle (the "noodle")
Drag it anywhere inside HomePage (Frame B)

A blue line will connect the button to the destination frame

Step 4: Configure the Interaction

A panel appears: "Interaction Details"

Trigger: "On click" (default, perfect)

Action: "Navigate to"

Destination: "HomePage" (auto-selected)

Animation: "Smart Animate" or "Push" (your choice)

Direction: "Right" (for natural flow)

Step 5: Test It

Select LoginPage frame

Click the Play button (▶) in top right

Your design opens in presentation mode

Click the button → Navigates to HomePage!

What's Happening Under the Hood:

Figma's prototype is a **preview**. The AI doesn't read prototypes directly. But prototypes serve two purposes:

1. **For you:** Test the flow and feel of your app
2. **For the AI:** The semantic structure of your components tells the AI what's interactive

When Antigravity reads your buttons, it sees:

- Button has **type: COMPONENT**
- Button inside a LoginPage frame
- Another frame below called HomePage

The AI infers: "This is a navigation button. Route clicking this button to the HomePage."

Scenario: Tab Navigation with Dynamic Highlighting

Step 1: Create Individual Tab Items

Build TabItem Component (as covered earlier):

- ❖ TabItem
 - | State: Inactive (gray icon, gray text)
 - | State: Active (blue icon, blue text, green dot behind)

Create three Instances on your screen:

- ❖ TabItem (label: "Home", state: Active)
- ❖ TabItem (label: "Search", state: Inactive)
- ❖ TabItem (label: "Profile", state: Inactive)

Step 2: Wrap in Tab Bar Component

Select all three tabs

Shift + A → Horizontal Auto Layout

Frame name: "TabBar" or "BottomNavigation"

Direction: Horizontal

Gap: 0 (tabs are adjacent)

Each tab horizontal resizing: Fill

This makes them share space evenly (33% each)

Step 3: Make Tab Bar a Component

Select the frame → Ctrl + Alt + K

Name it: "BottomNavigationBar" or "TabBar"

This semantic name tells the AI: "This is a navigation bar, not just a random grouped set of buttons."

Step 4: Create Variants for Each Active Tab

Click "Add Variant" button

Create variants:

- ❖ TabBar (activeTab: "Home")
 - ├─ ◇ TabItem (state: Active)
 - ├─ ◇ TabItem (state: Inactive)
 - └─ ◇ TabItem (state: Inactive)
- ❖ TabBar (activeTab: "Search")
 - ├─ ◇ TabItem (state: Inactive)
 - ├─ ◇ TabItem (state: Active)
 - └─ ◇ TabItem (state: Inactive)
- ❖ TabBar (activeTab: "Profile")
 - ├─ ◇ TabItem (state: Inactive)
 - ├─ ◇ TabItem (state: Inactive)
 - └─ ◇ TabItem (state: Active)

What the AI Sees:

```
{
  "name": "BottomNavigationBar",
  "type": "COMPONENT",
  "properties": [
    {
      "name": "activeTab",
      "type": "VARIANT",
      "values": ["Home", "Search", "Profile"]
    }
  ],
  "semanticType": "NAVIGATION_BAR"
}
```

The AI generates:

```

JS
const BottomNavigationBar = ({ activeTab = "Home", onTabChange }) => {
  const tabs = ["Home", "Search", "Profile"];

  return (
    <nav style={{ display: "flex" }}>
      {tabs.map((tab) => (
        <TabItem
          key={tab}
          label={tab}
          state={activeTab === tab ? "Active" : "Inactive"}
          onClick={() => onTabChange(tab)}
        />
      ))}
    </nav>
  );
};

```

Routing handled automatically. Perfect.

Designing for AI Code Generators

The Three Laws of AI-Ready Design

Law 1: Semantic Naming is Mandatory

Don't name things by what you see. Name them by what they are.

Bad Naming:

```

LoginPage
├─ Rectangle 1
├─ Group 2
├─ Text 3 ("Welcome Back")
├─ Frame 4
└─ Button_Click_Here

```

The AI tries to figure out what each element does and gives up. Result: messy, incorrect code.

Good Naming:

```

LoginPage
├── HeaderGroup
│   ├── TitleText ("Welcome Back")
│   └── SubtitleText ("Sign in to continue")
├── FormContainer
│   ├── ◊ TextInput (placeholder: "Email")
│   ├── ◊ TextInput (placeholder: "Password")
│   └── ◊ CheckboxField (label: "Remember me")
└── ActionContainer
    ├── ◊ PrimaryButton (label: "Sign In")
    └── ◊ TextButton (label: "Forgot password?")

```

The AI reads this tree and immediately knows the structure. Result: perfect code generation.

Naming Conventions for Different Elements:

Element Type	Naming Pattern	Example
Container	{Function}Container	FormContainer , HeaderGroup
Text (Heading)	TitleText , HeadingText	PageTitle , SectionHeading
Text (Body)	BodyText , DescriptionText	WelcomeMessage , ErrorText
Input	{Type}Input	EmailInput , PasswordInput
Button	{Type}Button or just semantic	PrimaryButton , SignInButton
Icon	{Name}Icon	GoogleIcon , BackIcon
Image	{Purpose}Image	HeroImage , BackgroundImage

Law 2: Always Use Instances, Never Main Components in Designs

Wrong:

```

LoginPage
├── ◊ Button (Main Component)
└── ◊ Button (Main Component)

```

The AI sees component definitions inline. Confusion.

Right:

Components Page:

└─ ♦ Button (Main Component)

LoginPage:

└─ ◇ Button (Instance)

└─ ◇ Button (Instance)

Clear separation. Components are library. Designs use the library.

Law 3: Auto Layout Everywhere (Except Vector Graphics)

Wrong:

Button

└─ Rectangle (X: 100, Y: 50, W: 200, H: 60)

└─ Text (X: 110, Y: 65)

Absolute positioning. The AI has to calculate and generate absolute CSS. Fragile.

Right:

Button (Auto Layout, Horizontal)

└─ Padding: 16px horizontal, 12px vertical

└─ Gap: 8px

└─ Text

The AI reads Auto Layout properties and generates flexbox. Responsive. Clean.

The Complete Semantic Structure Checklist

Before handing off to Antigravity, verify your file:

- ✓ All interactive elements (buttons, inputs) are Components
 - ✓ All components have semantic names (PrimaryButton, not Button_1)
 - ✓ All text that changes is a Text Property
 - ✓ All optional elements have Boolean Properties
 - ✓ All states have Variants (Default, Hover, Disabled, Error, etc.)
 - ✓ All components live on a dedicated Components/Design System page
 - ✓ All designs use Instances, not Main Components
 - ✓ Every container uses Auto Layout
 - ✓ No unnamed layers (no "Group 1", "Frame 2", "Text 3")
 - ✓ No absolute positioning except for complex vectors/logos
 - ✓ No hardcoded colors (use a color palette component or consistent fills)
-

Responsive Design Patterns

The Responsive Screen Template

Step 1: Set Up Your Main Frame

Press F → Create a frame sized to iPhone 15 Pro (390 x 844)

Name it: "LoginPage" or "Screen_Login"

Select the frame → Shift + A (Apply Auto Layout)

Direction: Vertical (top to bottom)

Padding: 24px horizontal (left and right)

Padding: 0px vertical (manage vertical spacing with gaps)

Step 2: Structure Major Sections

LoginPage (Auto Layout: Vertical)

- ├ HeaderSection
 - └ (Contents will be defined)

- ├ FormSection
 - └ (Contents will be defined)

- └ FooterSection
 - └ (Contents will be defined)

Step 3: Make Sections Auto Layout

```
HeaderSection (Auto Layout: Vertical)
├─ TitleText ("Welcome Back")
├─ SubtitleText ("Sign in to your account")
Properties:
  - Direction: Vertical
  - Gap: 8px (tight spacing for header)
  - Horizontal Resizing: Fill container
  - Vertical Resizing: Hug contents
```

```
FormSection (Auto Layout: Vertical)
├─ ◇ TextInput
├─ ◇ TextInput
├─ ◇ PrimaryButton
Properties:
  - Direction: Vertical
  - Gap: 16px (standard spacing for form)
  - Horizontal Resizing: Fill container
  - Vertical Resizing: Hug contents
```

```
FooterSection (Auto Layout: Horizontal)
├─ TextButton ("Don't have an account?")
├─ TextButton ("Create one")
Properties:
  - Direction: Horizontal
  - Gap: 4px (tight spacing for linked text)
  - Horizontal Resizing: Center
  - Vertical Resizing: Hug contents
```

Step 4: Handle Vertical Spacing (The Secret Sauce)

```
LoginPage (Auto Layout: Vertical)
├─ HeaderSection (Hug height)
├─ FormSection (Hug height)
├─ FooterSection (Hug height)
```

Now, if you want:

- Content at the top
- Form in the middle
- Footer at the bottom (even on tall screens)

Change LoginPage gap from fixed to "Auto":
Right panel → Auto Layout → Gap → Click dropdown → Select "Auto"

This creates `justify-content: space-between` in code.
Spacing distributes evenly, pushing items to edges.

Making Text Responsive

For Wrapping Text:

TitleText

- Font Size: 24px
- Text Resizing: "Auto height" (↕)
- Horizontal Resizing: Fill container

Result: Text takes 100% width and wraps to next line if needed.

For Fluid Sizing (Shrinking on Small Screens):

Figma doesn't have native fluid sizing, but you can hint to the AI:

Step 1: Rename text layer to "Title_Fluid" or "ResponsiveTitle"

Step 2: Set base size to 24px (phone size)

When the AI sees "Fluid" in the name, it generates:

```
font-size: clamp(1rem, 5vw, 2rem);
```

This makes the font scale between 1rem and 2rem depending on viewport width.

Mobile-First Responsive Checklist

- ✓ Screen frame is 390px wide (standard mobile)
- ✓ Main container has 24px horizontal padding
- ✓ All text elements have "Fill container" + "Auto height"
- ✓ All buttons have "Fill container" width (unless icon-only)
- ✓ All input fields have "Fill container" width
- ✓ Vertical gaps are at least 16px (touch-friendly)
- ✓ Major sections use "Hug contents" height
- ✓ Footer section has "Fixed" positioning or uses Auto gap
- ✓ No elements are hardcoded to fixed pixel widths
- ✓ Touch targets are minimum 48px × 48px

Bottom Navigation & Tab Systems

Complete Bottom Navigation Guide

Scenario: App with Home, Send, Profile Tabs

Step 1: Create the Individual Tab Item Component

Build TabItem:

- ❖ TabItem (Main Component)
 - ├ Auto Layout: Vertical
 - ├ Alignment: Center
 - ├ Gap: 4px
 - ├ Padding: 8px vertical
 - ├ Icon layer (named "IconSymbol")
 - └ Size: 24px × 24px
 - └ Text layer (named "Label")
 - └ Font: 12px, semibold

Properties:

- + Text Property "label" (link to Label layer)
- + Variant Property "state" with values: Inactive, Active

Step 2: Design Inactive and Active States

State: Inactive

- ├ Icon: Gray (#999999)
- ├ Text: Gray (#999999)
- └ Background: Transparent

State: Active

- ├ Icon: Blue (#007AFF)
- ├ Text: Blue (#007AFF)
- ├ Background: Light blue circle behind icon (optional)
- └ Green dot above icon (optional indicator)

Step 3: Create the Bottom Navigation Bar Component

Assemble three TabItem Instances:

- ◇ TabItem (label: "Home", state: Active)
- ◇ TabItem (label: "Send", state: Inactive)
- ◇ TabItem (label: "Profile", state: Inactive)

Select all three → Shift + A

Properties:

- Direction: Horizontal
- Gap: 0 (tabs are adjacent)
- Horizontal Resizing: Fill container
- Vertical Resizing: Hug contents

Wrap in a frame with background:

- Fill: White or dark gray
- Stroke: Subtle top border (optional)
- Corner Radius: 0 (full width bottom bar)

Step 4: Make It a Component

Select the bar → Ctrl + Alt + K
Name it: "BottomNavigationBar"

This semantic name is critical. The AI reads "BottomNavigationBar" and knows to generate native navigation routing code.

Step 5: Create Variants for Each Active Tab

Click "Add Variant"
Property: "activeTab"

Variant 1: activeTab: "Home"
Variant 2: activeTab: "Send"
Variant 3: activeTab: "Profile"

In each variant:

- Set corresponding TabItem to state: Active
- Set others to state: Inactive
- Move any visual indicators (green dot) to match

Step 6: Advanced - Add the "Scoop" Effect

For the modern notched design (bar with a cutout at the top):

Create two separate components:

- ❖ BottomBar (the dark background)
- ❖ SelectedTabIndicator (the floating pill/circle)

In the active variant, position the indicator to overlap:

- Select the indicator
- Use "Absolute Position" (top right of design panel)
- Position it so it extends above the bar

The AI will read the Absolute Position and generate:

```
position: absolute;  
bottom: 100%;
```

What the AI Generates:

```
const BottomNavigationBar = ({ activeTab = "Home", onNavigate }) => {  
  const tabs = ["Home", "Send", "Profile"];  
  
  return (  
    <nav style={{ display: "flex", width: "100%" }}>  
      {tabs.map((tab) => (  
        <TabItem  
          key={tab}  
          label={tab}  
          state={activeTab === tab ? "Active" : "Inactive"}  
          onClick={() => onNavigate(tab)}  
        />  
      ))}  
    </nav>  
  );  
};
```

Complete Execution Checklist

Pre-Design Phase

- ☐ Decide on screen sizes (iPhone 15 Pro: 390×844, iPad: 1024×1366, etc.)
- ☐ Define your color palette (create Color Palette component)
- ☐ Define typography (create Typography guide page)
- ☐ Sketch your information architecture (what screens do you need?)
- ☐ Plan your component library (buttons, inputs, cards, etc.)

Component Creation Phase

For each reusable element (Button, Input, Card, etc.):

- ☐ Design the default state
- ☐ Create Auto Layout structure
- ☐ Add semantic naming to all layers
- ☐ Make it a Main Component (**Ctrl + Alt + K**)
- ☐ Add Text Properties for dynamic content
- ☐ Add Boolean Properties for optional elements
- ☐ Create Variants for all states (Default, Hover, Disabled, Error, etc.)
- ☐ Move to Components page
- ☐ Document its usage (in a notes file or separate page)

Screen Design Phase

For each screen (LoginPage, HomePage, etc.):

- ☐ Create a new page
- ☐ Create main frame with correct dimensions
- ☐ Apply Auto Layout to main frame (Vertical, padding, gaps)
- ☐ Organize into semantic sections (HeaderSection, FormSection, etc.)
- ☐ Use Instances (not Main Components) for all interactive elements
- ☐ Set all text properties correctly on each Instance
- ☐ Verify all text elements have "Fill container" + "Auto height"
- ☐ Verify all inputs have "Fill container" width
- ☐ Verify all buttons have semantic names
- ☐ Check no elements are using absolute positioning (except logos/icons)
- ☐ Verify gaps are at least 16px (touch-friendly)

AI Preparation Phase

- ☐ Verify no layers named "Group 1", "Frame 2", "Text 3", etc.
- ☐ Verify all components are on the Components page
- ☐ Verify all designs use Instances, not Main Components
- ☐ Verify all interactive elements are Components
- ☐ Verify all text content is either:
 - Hardcoded in static text (like titles)
 - Linked to Text Properties (like button labels)
- ☐ Verify the file structure follows the standard template:
 - Cover page
 - Components page
 - Design System page
 - Wireframes page (optional)
 - WIP page
 - Ready for Dev page
 - Archive page
- ☐ Test your prototypes (click through flows)
- ☐ Do a final semantic review (read through your layer tree)

Handoff to AI/Developers

- ☐ Export the Figma file (share link or download)
- ☐ Include a README or design guide documenting:
 - Component list and usage
 - Color palette
 - Typography scale
 - Spacing/padding standards
 - Any special notes about responsive behavior
- ☐ Point out any custom implementations (complex animations, etc.)

☐ Flag any assets that need manual SVG export

Quick Reference: Common Mistakes and How to Avoid Them

Mistake	Why It's Bad	Solution
Using Main Component directly in designs	AI thinks component is defined inline	Always use Instances, keep Main Components on Components page
Not using Auto Layout	AI generates absolute positioning	Apply Auto Layout to every container
Naming layers "Rectangle 1", "Text 2"	AI can't understand intent	Use semantic names: <code>SubmitButton</code> , <code>EmailInput</code>
Hardcoding text that should be dynamic	Button always says "Login"	Use Text Properties for button labels, placeholders, etc.
Not creating Variants for states	AI doesn't know button should change on hover	Create Default, Hover, Disabled, Active variants
Using absolute positioning for layout	Layout breaks on different screen sizes	Use Auto Layout with Fill/Hug instead
Mixing components and regular frames	Confusing and error-prone	Be consistent: all interactive elements are components
Not organizing components on separate page	Cluttered file, hard to find blueprints	Move all Main Components to dedicated Components page
Creating 50 button components instead of one smart one	Code has 50 button classes instead of 1	Use Properties and Variants: one Button component, infinite variations
Leaving Main Components in designs	They can be accidentally deleted	Always use Instances in actual screen designs

Key Takeaways

1. **Figma reads data, not pixels.** Structure matters more than aesthetics. A badly named, poorly organized beautiful design generates worse code than a logically structured plain design.
2. **Auto Layout is flexbox.** Every container should use Auto Layout to tell the AI how elements relate to each other.
3. **Components are functions.** Main Components are function definitions. Instances

are function calls. Always separate them.

4. **Properties are parameters.** Text properties, boolean properties, and instance swap properties make components flexible and reusable.
 5. **Variants are states.** Every interactive element should have Variants for Default, Hover, Disabled, Error, etc.
 6. **Semantic naming is everything.** `LoginButton` tells the AI what it is. `Button` tells it nothing.
 7. **Instances > Main Components.** In your designs, always drag from Assets. Never use the Main Component directly.
 8. **Responsive is achievable.** Use Fill/Hug resizing rules instead of fixed pixel widths.
 9. **Organize with Pages.** Keep Components separate, organize designs logically, archive old work.
 10. **Test before handing off.** Use Figma's Prototype feature to verify your flows. Fix semantic issues before showing the AI.
-

Additional Resources

Color & Typography

When creating a design system, always create components for:

Colors:

- ❖ ColorPalette (not strictly a component, but a reference page)
 - Primary: #007AFF
 - Secondary: #5AC8FA
 - Error: #FF3B30
 - Success: #34C759
 - Gray 100: #F5F5F5
 - Gray 500: #999999
 - Gray 900: #333333

Typography:

- ❖ HeadingLarge (24px, semibold)
- ❖ HeadingMedium (18px, semibold)
- ❖ BodyText (16px, regular)
- ❖ CaptionText (12px, regular)
- ❖ ButtonText (16px, semibold)

These aren't interactive components, but create them so you're consistent.

Spacing Scale

Define and use a spacing scale:

```
Spacing increments:  
4px  (xs: tight spacing)  
8px  (sm: compact spacing)  
12px (base: standard spacing)  
16px (md: comfortable spacing)  
24px (lg: generous spacing)  
32px (xl: large section spacing)
```

When creating Auto Layout, always use these standard values, not arbitrary numbers.

Touch Targets

Remember mobile usability:

```
Minimum touch target: 48px × 48px  
Button minimum: 48px height  
Input field minimum: 48px height  
Icon button minimum: 48px × 48px  
Text button minimum: 40px height
```

Verify in your designs that all interactive elements meet these minimums.

Conclusion

You now have the complete blueprint for creating Figma files that:

1. Are beautiful and functional as designs
2. Are perfectly structured for AI code generators
3. Teach developers exactly what you intended
4. Scale to responsive layouts
5. Maintain consistency across your entire product
6. Are easy to maintain and iterate

The key is understanding that Figma is both a design tool and a data specification tool. The better your data structure, the better the code that comes out the other end.

Start with a simple screen (LoginPage with button, input, text). Master that. Then scale to more complex screens. Your third or fourth screen will take half the time of your first

because you'll have your component library and spacing system refined.

Good luck designing!